

Studento klases

Sugeneruota Doxygen 1.17.0

1 Direktorijų hierarchija	1
1.1 Direktorijos	1
2 Hierarchijos Indeksas	3
2.1 Klasių hierarchija	3
3 Klasės Indeksas	5
3.1 Klasės	5
4 Failo Indeksas	7
4.1 Failai	7
5 Direktorijų dokumentacija	9
5.1 include Direktorijos aprašas	9
5.2 src Direktorijos aprašas	9
6 Klasės Dokumentacija	11
6.1 Studentas Klasė	11
6.1.1 Smulkus aprašymas	12
6.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	12
6.1.2.1 Studentas() [1/3]	12
6.1.2.2 Studentas() [2/3]	13
6.1.2.3 Studentas() [3/3]	13
6.1.2.4 ~Studentas()	13
6.1.3 Metodų Dokumentacija	13
6.1.3.1 getEgz()	13
6.1.3.2 getPaz()	14
6.1.3.3 isvalyti_pazymius()	14
6.1.3.4 nuskaityti_is_eilutes()	14
6.1.3.5 nuskaityti_ranka()	14
6.1.3.6 operator=() [1/2]	14
6.1.3.7 operator=() [2/2]	15
6.1.3.8 readStudent()	15
6.1.3.9 Rezultatas()	15
6.1.3.10 setEgz()	15
6.1.3.11 setPavarde()	15
6.1.3.12 setPaz()	16
6.1.3.13 setVardas()	16
6.1.3.14 skaiciuoti_rezultata()	16
6.1.3.15 whoAml() [1/2]	16
6.1.3.16 whoAml() [2/2]	16
6.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	17
6.1.4.1 operator<< [1/2]	17
6.1.4.2 operator<< [2/2]	17

6.1.4.3 operator>>	17
6.2 Zmogus Klasė	17
6.2.1 Smulkus aprašymas	18
6.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	18
6.2.2.1 Zmogus()	18
6.2.2.2 ~Zmogus()	18
6.2.3 Metodų Dokumentacija	18
6.2.3.1 getPavarde()	18
6.2.3.2 getVardas()	19
6.2.3.3 whoAml() [1/2]	19
6.2.3.4 whoAml() [2/2]	19
6.2.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	19
6.2.4.1 operator<<	19
6.2.5 Atributų Dokumentacija	19
6.2.5.1 Pavarde	19
6.2.5.2 Vardas	19
7 Failo Dokumentacija	21
7.1 include/bibliotekos.h Failo Nuoroda	21
7.2 bibliotekos.h	21
7.3 include/funkcijos.h Failo Nuoroda	22
7.3.1 Funkcijos Dokumentacija	22
7.3.1.1 bufer_nusk()	22
7.3.1.2 failuGeneravimas()	23
7.3.1.3 generavimasSk()	23
7.3.1.4 generavimasVisko()	23
7.3.1.5 inputas()	23
7.3.1.6 outputas()	24
7.3.1.7 rikiavimas()	24
7.3.1.8 skaiciu_mastelis()	24
7.3.1.9 skaiciu_skaitymas()	25
7.3.1.10 studentoLygis()	25
7.3.1.11 tyrimasFailoKurimas()	25
7.3.1.12 tyrimasKlasesMetodams()	25
7.3.1.13 tyrimasVisasProcesas()	26
7.3.1.14 vardo_skaitymas()	26
7.4 funkcijos.h	26
7.5 include/studentas.h Failo Nuoroda	27
7.5.1 Tipų apibrėžimų Dokumentacija	27
7.5.1.1 StudentuGrupe	27
7.5.2 Funkcijos Dokumentacija	27
7.5.2.1 skaiciu_mastelis()	27

7.5.2.2 vardo_skaitymas()	28
7.5.3 Kintamojo Dokumentacija	28
7.5.3.1 Maxpazymiu	28
7.5.3.2 Maxstudentu	28
7.5.3.3 studentas	28
7.6 studentas.h	29
7.7 src/generavimas.cpp Failo Nuoroda	30
7.7.1 Funkcijos Dokumentacija	30
7.7.1.1 failuGeneravimas()	30
7.7.1.2 generavimasSk()	30
7.7.1.3 generavimasVisko()	31
7.8 generavimas.cpp	31
7.9 src/ived_isved.cpp Failo Nuoroda	33
7.9.1 Funkcijos Dokumentacija	33
7.9.1.1 bufer_nusk()	33
7.9.1.2 inputas()	33
7.9.1.3 outputas()	34
7.10 ived_isved.cpp	34
7.11 src/main.cpp Failo Nuoroda	36
7.11.1 Funkcijos Dokumentacija	37
7.11.1.1 main()	37
7.12 main.cpp	37
7.13 src/papildomos_funkcijos.cpp Failo Nuoroda	38
7.13.1 Funkcijos Dokumentacija	38
7.13.1.1 rikiavimas()	38
7.13.1.2 skaiciu_mastelis()	39
7.13.1.3 skaiciu_skaitymas()	39
7.13.1.4 studentoLygis()	39
7.13.1.5 vardo_skaitymas()	39
7.14 papildomos_funkcijos.cpp	40
7.15 src/studentas.cpp Failo Nuoroda	42
7.15.1 Funkcijos Dokumentacija	42
7.15.1.1 operator<<() [1/2]	42
7.15.1.2 operator<<() [2/2]	42
7.15.1.3 operator>>()	43
7.16 studentas.cpp	43
7.17 src/tyrimai.cpp Failo Nuoroda	45
7.17.1 Funkcijos Dokumentacija	45
7.17.1.1 tyrimasFailoKurimas()	45
7.17.1.2 tyrimasKlasesMetodams()	46
7.17.1.3 tyrimasVisasProcesas()	46
7.18 tyrimai.cpp	46

skyrius 1

Direktorių hierarchija

1.1 Direktorijos

include	9
bibliotekos.h	21
funkcijos.h	22
studentas.h	27
src	9
generavimas.cpp	30
ived_isved.cpp	33
main.cpp	36
papildomos_funkcijos.cpp	38
studentas.cpp	42
tyrimai.cpp	45

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Zmogus	17
Studentas	11

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	11
Zmogus	17

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

include/bibliotekos.h	21
include/funkcijos.h	22
include/studentas.h	27
src/generavimas.cpp	30
src/ived_isved.cpp	33
src/main.cpp	36
src/papildomos_funkcijos.cpp	38
src/studentas.cpp	42
src/tyrimai.cpp	45

skyrius 5

Direktorių dokumentacija

5.1 include Direktorijos aprašas

Failai

- failas [bibliotekos.h](#)
- failas [funkcijos.h](#)
- failas [studentas.h](#)

5.2 src Direktorijos aprašas

Directory dependency graph for src:

Failai

- failas [generavimas.cpp](#)
- failas [ived_isved.cpp](#)
- failas [main.cpp](#)
- failas [papildomos_funkcijos.cpp](#)
- failas [studentas.cpp](#)
- failas [tyrimai.cpp](#)

skyrius 6

Klasės Dokumentacija

6.1 Studentas Klasė

#include <studentas.h>

Paveldimumo diagrama Studentas:

Bendradarbiavimo diagrama Studentas:

Vieši Metodai

- `Studentas ()`
konstruktoriaus realizacija
- void `whoAml () const`
default konstruktorius
- virtual std::ostream & `whoAml (std::ostream &out) const`
- const vector< int > & `getPaz () const`
- int `getEgz () const`
getteris
- double `Rezultatas () const`
getteris
- std::istream & `readStudent (std::istream &)`
getteris
- void `setVardas (const string &v)`
setteris
- void `setPavarde (const string &p)`
- void `setPaz (int p)`
- void `setEgz (int e)`
- `Studentas (const Studentas &s)`
- `Studentas (Studentas &&s)`
copy konstruktorius
- `Studentas & operator= (const Studentas &s)`
move konstruktorius
- `Studentas & operator= (Studentas &&s)`
copy priskyrimas =
- `~Studentas ()`
move priskyrimas =
- void `nuskaityti_ranka (int max_pazymiu)`
destruktorius
- void `nuskaityti_is_eilutes (std::istream &is)`
- void `skaiciuoti_rezultata (int pasirinkimas)`
- void `isvalyti_pazymius ()`

Vieši Metodai inherited from **Zmogus**

- virtual string [getVardas](#) () const
- virtual string [getPavarde](#) () const
- virtual [~Zmogus](#) ()

Draugai

- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)
isvedimo operatorius
- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
ivedimo operatorius
- std::ofstream & [operator<<](#) (std::ofstream &os, const [Studentas](#) &s)
isvedimo operatorius i faila

Additional Inherited Members

Apsaugoti Metodai inherited from **Zmogus**

- [Zmogus](#) (string v="", string p="")

Apsaugoti Atributai inherited from **Zmogus**

- string [Vardas](#)
- string [Pavarde](#)

6.1.1 Smulkus aprašymas

Apibrėžimas failo [studentas.h](#) eilutėje 36.

6.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

6.1.2.1 Studentas() [1/3]

```
Studentas::Studentas ()
```

konstruktoriaus realizacija

Apibrėžimas failo [studentas.cpp](#) eilutėje 4.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

6.1.2.2 Studentas() [2/3]

```
Studentas::Studentas (
    const Studentas & s)
```

copy konstruktorius

1. isskiriama nauja vieta
2. perkopijuoja reiksmes is v

Apibrėžimas failo [studentas.cpp](#) eilutėje 24.

Funkcijos kvietimo grafas:

6.1.2.3 Studentas() [3/3]

```
Studentas::Studentas (
    Studentas && s)
```

copy konstruktorius

move konstruktorius 1."pavagiame" reiksmes is s

Apibrėžimas failo [studentas.cpp](#) eilutėje 39.

Funkcijos kvietimo grafas:

6.1.2.4 ~Studentas()

```
Studentas::~~Studentas ()
```

move priskyrimas =

destruktoriaus realizacija

Apibrėžimas failo [studentas.cpp](#) eilutėje 15.

6.1.3 Metodų Dokumentacija

6.1.3.1 getEgz()

```
int Studentas::getEgz () const [inline]
```

getteris

Apibrėžimas failo [studentas.h](#) eilutėje 58.

Here is the caller graph for this function:

6.1.3.2 getPaz()

```
const vector< int > & Studentas::getPaz () const [inline]
```

Apibrėžimas failo [studentas.h](#) eilutėje 57.

6.1.3.3 isvalyti_pazymius()

```
void Studentas::isvalyti_pazymius ()
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 169.

Here is the caller graph for this function:

6.1.3.4 nuskaityti_is_eilutes()

```
void Studentas::nuskaityti_is_eilutes (  
    std::istream & is)
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 121.

6.1.3.5 nuskaityti_ranka()

```
void Studentas::nuskaityti_ranka (  
    int max_pazymiu)
```

destruktorius

Apibrėžimas failo [studentas.cpp](#) eilutėje 97.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

6.1.3.6 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & s)
```

move konstruktorius

copy proskyrimas

Apibrėžimas failo [studentas.cpp](#) eilutėje 60.

Funkcijos kvietimo grafas:

6.1.3.7 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && s)
```

copy priskyrimas =

move priskyrimas

Apibrėžimas failo [studentas.cpp](#) eilutėje 74.

Funkcijos kvietimo grafas:

6.1.3.8 readStudent()

```
std::istream & Studentas::readStudent (
    std::istream & )
```

getteris

6.1.3.9 Rezultatas()

```
double Studentas::Rezultatas () const [inline]
```

getteris

Apibrėžimas failo [studentas.h](#) eilutėje 59.

Here is the caller graph for this function:

6.1.3.10 setEgz()

```
void Studentas::setEgz (
    int e) [inline]
```

Apibrėžimas failo [studentas.h](#) eilutėje 71.

Here is the caller graph for this function:

6.1.3.11 setPavarde()

```
void Studentas::setPavarde (
    const string & p) [inline]
```

Apibrėžimas failo [studentas.h](#) eilutėje 65.

Here is the caller graph for this function:

6.1.3.12 setPaz()

```
void Studentas::setPaz (
    int p) [inline]
```

Apibrėžimas failo [studentas.h](#) eilutėje 68.

Here is the caller graph for this function:

6.1.3.13 setVardas()

```
void Studentas::setVardas (
    const string & v) [inline]
```

setteris

Apibrėžimas failo [studentas.h](#) eilutėje 62.

Here is the caller graph for this function:

6.1.3.14 skaiciuoti_rezultata()

```
void Studentas::skaiciuoti_rezultata (
    int pasirinkimas)
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 136.

Here is the caller graph for this function:

6.1.3.15 whoAmI() [1/2]

```
void Studentas::whoAmI () const [inline], [virtual]
```

default konstruktorius

Realizuoja [Zmogus](#).

Apibrėžimas failo [studentas.h](#) eilutėje 51.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

6.1.3.16 whoAmI() [2/2]

```
virtual std::ostream & Studentas::whoAmI (
    std::ostream & out) const [inline], [virtual]
```

Perkrauna metodą iš [Zmogus](#).

Apibrėžimas failo [studentas.h](#) eilutėje 52.

Funkcijos kvietimo grafas:

6.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

6.1.4.1 operator<< [1/2]

```
std::ofstream & operator<< (  
    std::ofstream & os,  
    const Studentas & s) [friend]
```

isvedimo operatorius i faila

Apibrėžimas failo [studentas.cpp](#) eilutėje 182.

6.1.4.2 operator<< [2/2]

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s) [friend]
```

isvedimo operatorius

Apibrėžimas failo [studentas.cpp](#) eilutėje 174.

6.1.4.3 operator>>

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s) [friend]
```

ivedimo operatorius

Apibrėžimas failo [studentas.cpp](#) eilutėje 190.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- include/[studentas.h](#)
- src/[studentas.cpp](#)

6.2 Zmogus Klasė

```
#include <studentas.h>
```

Paveldimumo diagrama Zmogus:

Vieši Metodai

- virtual string [getVardas](#) () const
- virtual string [getPavarde](#) () const
- virtual void [whoAmI](#) () const =0
- *virtual'i funkcija*
- virtual std::ostream & [whoAmI](#) (std::ostream &out) const
- virtual [~Zmogus](#) ()

Apsaugoti Metodai

- [Zmogus](#) (string v="", string p="")

Apsaugoti Atributai

- string [Vardas](#)
- string [Pavarde](#)

Draugai

- std::ostream & [operator<<](#) (std::ostream &out, const [Zmogus](#) &z)
Overloadint'as operator<< kaip friend funkcija, o dešininis operandas yra Base&.

6.2.1 Smulkus aprašymas

Apibrėžimas failo [studentas.h](#) eilutėje 9.

6.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

6.2.2.1 Zmogus()

```
Zmogus::Zmogus (
    string v = "",
    string p = "") [inline], [protected]
```

Apibrėžimas failo [studentas.h](#) eilutėje 13.

Here is the caller graph for this function:

6.2.2.2 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Apibrėžimas failo [studentas.h](#) eilutėje 31.

6.2.3 Metodų Dokumentacija

6.2.3.1 getPavarde()

```
virtual string Zmogus::getPavarde () const [inline], [virtual]
```

Apibrėžimas failo [studentas.h](#) eilutėje 17.

Here is the caller graph for this function:

6.2.3.2 getVardas()

```
virtual string Zmogus::getVardas () const [inline], [virtual]
```

Apibrėžimas failo [studentas.h](#) eilutėje 16.

Here is the caller graph for this function:

6.2.3.3 whoAmI() [1/2]

```
virtual void Zmogus::whoAmI () const [pure virtual]
```

virtual'i funkcija

Realizuota [Studentas](#).

Here is the caller graph for this function:

6.2.3.4 whoAmI() [2/2]

```
virtual std::ostream & Zmogus::whoAmI (  
    std::ostream & out) const [inline], [virtual]
```

Metodas perkraunamas [Studentas](#).

Apibrėžimas failo [studentas.h](#) eilutėje 22.

6.2.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

6.2.4.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & out,  
    const Zmogus & z) [friend]
```

Overloadint'as operator<< kaip friend funkcija, o dešininis operandas yra Base&.

Apibrėžimas failo [studentas.h](#) eilutėje 27.

6.2.5 Atributų Dokumentacija

6.2.5.1 Pavarde

```
string Zmogus::Pavarde [protected]
```

Apibrėžimas failo [studentas.h](#) eilutėje 12.

6.2.5.2 Vardas

```
string Zmogus::Vardas [protected]
```

Apibrėžimas failo [studentas.h](#) eilutėje 11.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- include/[studentas.h](#)

skyrius 7

Failo Dokumentacija

7.1 include/bibliotekos.h Failo Nuoroda

```
#include <iostream>
#include <string>
#include <vector>
#include <list>
#include <deque>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <cctype>
#include <random>
#include <chrono>
#include <stdlib.h>
#include <fstream>
#include <sstream>
#include <cstdlib>
#include <cstdio>
#include <stdexcept>
```

Įtraukimo priklausomybių diagrama bibliotekos.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

7.2 bibliotekos.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #pragma once
00002 #include <iostream>
00003 #include <string>
00004 #include <vector>
00005 #include <list>
00006 #include <deque>
00007 #include <iomanip>
00008 #include <algorithm>
00009 #include <limits>
00010 #include <cctype>
00011 #include <random>
00012 #include <chrono>
00013 #include <stdlib.h>
00014 #include <fstream>
00015 #include <sstream>
00016 #include <cstdlib>
00017 #include <cstdio>
```

```

00018 #include <limits>
00019 #include <stdexcept>
00020 using std::string;
00021 using std::cout;
00022 using std::cin;
00023 using std::endl;
00024 using std::left;
00025 using std::right;
00026 using std::setw;
00027 using std::vector;
00028 using std::fixed;
00029 using std::setprecision;
00030 using std::mt19937;
00031 using std::uniform_int_distribution;
00032 using std::chrono::high_resolution_clock;
00033 using std::sort;

```

7.3 include/funkcijos.h Failo Nuoroda

```
#include "studentas.h"
```

Įtraukimo priklausomybių diagrama funkcijos.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Funkcijos

- void [inputas](#) ([Studentas](#) &A, [StudentuGrupe](#) &grupe, int &pasirinkimas)
- void [outputas](#) (const [StudentuGrupe](#) &vargsiukai, const [StudentuGrupe](#) &smartukai, int &pasirinkimas, int &isvedimas, int &m)
- [StudentuGrupe](#) [bufer_nusk](#) (string &read_vardas, int &pasirinkimas, int &m)
- void [generavimasSk](#) ([Studentas](#) &A, [StudentuGrupe](#) &grupe, int &pasirinkimas)
- void [generavimasVisko](#) ([Studentas](#) &A, [StudentuGrupe](#) &grupe, int &pasirinkimas)
- void [failuGeneravimas](#) (int &n)
- int [skaiciu_mastelis](#) (const string &prompt, int min_val, int max_val)
- string [vardo_skaitymas](#) (const string &prompt)
- double [skaiciu_skaitymas](#) (const string &prompt, double min_val, double max_val)
- void [rikiavimas](#) ([StudentuGrupe](#) &grupe, int &pasirinkimas)
Funkcija rikiavimui, kuri naudoja lambda funkcijas kaip comparatorius, kad galėtų rikiuoti pagal vardą, pavardę arba galutinį balą, priklausomai nuo vartotojo pasirinkimo.
- void [studentoLygis](#) ([StudentuGrupe](#) &grupe, [StudentuGrupe](#) &vargsiukai, [StudentuGrupe](#) &smartukai, int &rusiavimas)
- void [tyrimasFailoKurimas](#) ()
- void [tyrimasVisasProcesas](#) ()
- void [tyrimasKlasesMetodams](#) ()

7.3.1 Funkcijos Dokumentacija

7.3.1.1 bufer_nusk()

```

StudentuGrupe bufer_nusk (
    string & read_vardas,
    int & pasirinkimas,
    int & m)

```

Sukuriame stringstream objektą, kuris leis mums lengvai išskaidyti eilutę į atskirus žodžius ir skaičius

Skaitome eilutę į [Studentas](#) objektą per klasės metodą

Išvalome pažymių vektorių, kad jis būtų tuščias prieš kitą studento įvedimą

Apibrėžimas failo [ived_isved.cpp](#) eilutėje 4.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.2 failuGeneravimas()

```
void failuGeneravimas (  
    int & n)
```

Apibrėžimas failo [generavimas.cpp](#) eilutėje 73.

Here is the caller graph for this function:

7.3.1.3 generavimasSk()

```
void generavimasSk (  
    Studentas & A,  
    StudentuGrupe & grupe,  
    int & pasirinkimas)
```

Apibrėžimas failo [generavimas.cpp](#) eilutėje 3.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.4 generavimasVisko()

```
void generavimasVisko (  
    Studentas & A,  
    StudentuGrupe & grupe,  
    int & pasirinkimas)
```

Inicijuojame atsitiktinių skaičių generatorių su dabartiniu laiku

`static_cast<long unsigned int>` naudojamas norint užtikrinti, kad laiko reikšmė būtų tinkamai konvertuota į generatoriaus sėklą

Sukuriame tolygų skaičių pasiskirstymą nuo 0 iki 9

Apibrėžimas failo [generavimas.cpp](#) eilutėje 35.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.5 inputas()

```
void inputas (  
    Studentas & A,  
    StudentuGrupe & grupe,  
    int & pasirinkimas)
```

Rankinis įvedimas per klasės metodą

Pridedame studentą į grupę

Išvalome pažymių vektorius, kad jis būtų tuščias prieš kitą studento įvedimą

Apibrėžimas failo [ived_isved.cpp](#) eilutėje 60.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.6 outputas()

```
void outputas (
    const StudentuGrupe & vargsiukai,
    const StudentuGrupe & smartukai,
    int & pasirinkimas,
    int & isvedimas,
    int & m)
```

Pradedame matuoti laiką

`std::chrono::high_resolution_clock::now()` funkcija grąžina dabartinį laiką, kuris bus naudojamas vėliau apskaičiuoti, kiek laiko užtruko failo nuskaitymas ir apdorojimas

Apskaičiuojame, kiek laiko praėjo nuo pradžios iki dabar, ir išsaugome šį laiką `diff` kintamajame

`std::chrono::duration<double>` yra tipas, kuris saugo laiką sekundėmis kaip `double` reikšmę, o `diff.count()` grąžina šią reikšmę, kurią mes išvedame į ekraną

Apskaičiuojame, kiek laiko praėjo nuo pradžios iki dabar, ir išsaugome šį laiką `diff` kintamajame

`std::chrono::duration<double>` yra tipas, kuris saugo laiką sekundėmis kaip `double` reikšmę, o `diff.count()` grąžina šią reikšmę, kurią mes išvedame į ekraną

Apibrėžimas failo [ived_isved.cpp](#) eilutėje 74.

Here is the caller graph for this function:

7.3.1.7 rikiavimas()

```
void rikiavimas (
    StudentuGrupe & grupe,
    int & pasirinkimas)
```

Funkcija rikiavimui, kuri naudoja lambda funkcijas kaip comparatorius, kad galėtų rikiuoti pagal vardą, pavardę arba galutinį balą, priklausomai nuo vartotojo pasirinkimo.

`list` naudos `list::sort()`, o `vector/deque` naudos `std::sort()`

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 14.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.8 skaiciu_mastelis()

```
int skaiciu_mastelis (
    const string & prompt,
    int min_val,
    int max_val)
```

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 30.

Here is the caller graph for this function:

7.3.1.9 skaiciu_skaitymas()

```
double skaiciu_skaitymas (
    const string & prompt,
    double min_val,
    double max_val)
```

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 156.

7.3.1.10 studentoLygis()

```
void studentoLygis (
    StudentuGrupe & grupe,
    StudentuGrupe & vargsiukai,
    StudentuGrupe & smartukai,
    int & rusiavimas)
```

1 strategija: visi studentai lieka grupes, kopijuojami į vargsiukai ir smartukai

2 strategija: vienu perėjimu vargsiukus perkeliame, o iš grupes ištriname.

3 strategija: dalijame su partition, vargsiukus perkeliame, grupėje paliekame tik smartukus.

Kopijuojame vargsiukus į atskirą konteinerį, naudojant std::copy ir std::back_inserter, kuris prideda elementus į vargsiukai konteinerio pabaigą.

Iš grupės ištriname vargsiukus, palikdami tik smartukus.

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 65.

Here is the caller graph for this function:

7.3.1.11 tyrimasFailoKurimas()

```
void tyrimasFailoKurimas ()
```

Apibrėžimas failo [tyrimai.cpp](#) eilutėje 57.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.12 tyrimasKlasesMetodams()

```
void tyrimasKlasesMetodams ()
```

1. Copy konstruktoriaus testas

1. Copy priskyrimo operatoriaus testas

1. Move konstruktoriaus testas

1. Move priskyrimo operatoriaus testas

Apibrėžimas failo [tyrimai.cpp](#) eilutėje 3.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.13 tyrimasVisasProcesas()

```
void tyrimasVisasProcesas ()
```

Apibrėžimas failo [tyrimai.cpp](#) eilutėje 72.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.3.1.14 vardo_skaitymas()

```
string vardo_skaitymas (
    const string & prompt)
```

Kintamasis, kuris nurodo, ar įvestyje yra bent viena raidė

Kintamasis, kuris nurodo, ar visi įvesties simboliai yra raidės arba skaičiai

std::isalpha funkcija tikrina, ar simbolis yra raidė

std::isalnum funkcija tikrina, ar simbolis yra raidė arba skaičius

std::numeric_limits<std::streamsize>::max() - tai funkcija, kuri grąžina didžiausią galimą streamsize reikšmę, kuri yra naudojama kaip argumentas ignore funkcijai, kad būtų ignoruojami visi likę simboliai įvesties sraute iki naujos eilutės simbolio ('').

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 105.

Here is the caller graph for this function:

7.4 funkcijos.h

Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002 #include "studentas.h"
00003
00004
00005
00006 void inputas(Studentas &A, StudentuGrupe &grupe, int &pasirinkimas);
00007 void outputas(const StudentuGrupe &vargsiukai, const StudentuGrupe &smartukai, int &pasirinkimas, int
    &isvedimas, int &m);
00008 StudentuGrupe bufer_nusk(string &read_vardas, int &pasirinkimas, int &m);
00009 void generavimasSk(Studentas &A, StudentuGrupe &grupe, int &pasirinkimas);
00010 void generavimasVisko(Studentas &A, StudentuGrupe &grupe, int &pasirinkimas);
00011 void failuGeneravimas(int &n);
00012
00013 int skaiciu_mastelis(const string &prompt, int min_val, int max_val);
00014 string vardo_skaitymas(const string &prompt);
00015 double skaiciu_skaitymas(const string &prompt, double min_val, double max_val);
00016 void rikiavimas(StudentuGrupe &grupe, int &pasirinkimas);
00017 void studentoLygis(StudentuGrupe &grupe, StudentuGrupe &vargsiukai, StudentuGrupe &smartukai, int
    &rusiavimas);
00018 void tyrimasFailoKurimas();
00019 void tyrimasVisasProcesas();
00020 void tyrimasKlasesMetodams();
```

7.5 include/studentas.h Failo Nuoroda

```
#include "bibliotekos.h"
```

Įtraukimo priklausomybių diagrama studentas.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Klasės

- class [Zmogus](#)
- class [Studentas](#)

Tipų apibrėžimai

- using [StudentuGrupe](#) = std::vector<[Studentas](#)>
sukurti alias [StudentuGrupe](#), kuri galima naudoti kaip vektoriu, lista arba deque tipo konteineri, tam, kad patikrinti programos sparta, su skirtingo tipo konteineriais.

Funkcijos

- int [skaiciu_mastelis](#) (const string &prompt, int min_val, int max_val)
- string [vardo_skaitymas](#) (const string &prompt)

Kintamieji

- !Klase [studentas](#)
- const int [Maxpazymiu](#) =20
- const int [Maxstudentu](#) =10000000

7.5.1 Tipų apibrėžimų Dokumentacija

7.5.1.1 StudentuGrupe

```
using StudentuGrupe = std::vector<Studentas>
```

sukurti alias [StudentuGrupe](#), kuri galima naudoti kaip vektoriu, lista arba deque tipo konteineri, tam, kad patikrinti programos sparta, su skirtingo tipo konteineriais.

Apibrėžimas failo [studentas.h](#) eilutėje 96.

7.5.2 Funkcijos Dokumentacija

7.5.2.1 skaiciu_mastelis()

```
int skaiciu_mastelis (
    const string & prompt,
    int min_val,
    int max_val)
```

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 30.

Here is the caller graph for this function:

7.5.2.2 vardo_skaitymas()

```
string vardo_skaitymas (  
    const string & prompt)
```

Kintamasis, kuris nurodo, ar įvestyje yra bent viena raidė

Kintamasis, kuris nurodo, ar visi įvesties simboliai yra raidės arba skaičiai

std::isalpha funkcija tikrina, ar simbolis yra raidė

std::isalnum funkcija tikrina, ar simbolis yra raidė arba skaičius

std::numeric_limits<std::streamsize>::max() - tai funkcija, kuri grąžina didžiausią galimą streamsize reikšmę, kuri yra naudojama kaip argumentas ignore funkcijai, kad būtų ignoruojami visi likę simboliai įvesties sraute iki naujos eilutės simbolio ('').

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 105.

Here is the caller graph for this function:

7.5.3 Kintamojo Dokumentacija

7.5.3.1 Maxpazymiu

```
const int Maxpazymiu =20
```

Apibrėžimas failo [studentas.h](#) eilutėje 99.

7.5.3.2 Maxstudentu

```
const int Maxstudentu =10000000
```

Apibrėžimas failo [studentas.h](#) eilutėje 100.

7.5.3.3 studentas

```
!Klase studentas
```

Apibrėžimas failo [studentas.h](#) eilutėje 35.

7.6 studentas.h

Eiti į šio failo dokumentaciją.

```

00001
00002 #pragma once
00003 #include "bibliotekos.h"
00004
00005 int skaiciu_mastelis(const string &prompt, int min_val, int max_val);
00006 string vardo_skaitymas(const string &prompt);
00007
00008 /* Pagrindine klase zmogus
00009 class Zmogus {
00010     protected:
00011         string Vardas;
00012         string Pavarde;
00013     Zmogus(string v = "", string p = "") : Vardas{v}, Pavarde{p} { Vardas=v; Pavarde=p;
00014         /*std::cout<<"Zmogus K.\n";*/ }
00015
00016     public:
00017     virtual string getVardas() const { return Vardas; }
00018     virtual string getPavarde() const { return Pavarde; }
00019
00020     virtual void whoAmI() const = 0;
00021     //1 Overloadint'a (kita) virtuali whoAmI() funkcija
00022     virtual std::ostream& whoAmI(std::ostream& out) const {
00023         out << "Aš esu " << Vardas << " iš Base klasės\n";
00024         return out;
00025     }
00026     friend std::ostream& operator<<(std::ostream& out, const Zmogus &z) {
00027         // Visa darba atliks whoAmI() funkcija, kuri yra virtuali!
00028         return z.whoAmI(out);
00029     }
00030 }
00031 virtual ~Zmogus() {Vardas = ""; Pavarde = ""};
00032 };
00033
00034
00035 /* Klase studentas, kuri yra paveldeta is zmogaus klases
00036 class Studentas : public Zmogus {
00037     private:
00038         //string Vardas;
00039         //string Pavarde;
00040         vector<int> paz;
00041         int egz;
00042         double vidurkis;
00043         double mediana;
00044         double rez;
00045         string lygis;
00046     public:
00047
00048
00049     Studentas();
00050
00051     void whoAmI() const { std::cout << "Aš esu " << getVardas() << " iš Studentas klasės\n"; }
00052     virtual std::ostream& whoAmI(std::ostream& out) const {
00053         out << "Aš esu " << getVardas() << " iš Studentas klasės\n";
00054         return out;
00055     }
00056
00057     const vector<int>& getPaz() const { return paz; }
00058     int getEgz() const { return egz; }
00059     double Rezultatas() const {return rez;}
00060     std::istream& readStudent(std::istream&);
00061
00062     void setVardas(const string& v) {
00063         Vardas = v;
00064     }
00065     void setPavarde(const string& p) {
00066         Pavarde = p;
00067     }
00068     void setPaz(int p) {
00069         paz.push_back(p);
00070     }
00071     void setEgz(int e) {
00072         egz = e;
00073     }
00074
00075
00076     Studentas(const Studentas& s);
00077     Studentas(Studentas&& s);
00078     Studentas& operator=(const Studentas& s);
00079     Studentas& operator=(Studentas&& s);
00080     ~Studentas();
00081
00082
00083     void nuskaityti_ranka(int max_pazymiu);

```

```

00084 void nuskaityti_is_eilutes(std::istream& is);
00085 void skaiciuoti_rezultata(int pasirinkimas);
00086 void isvalyti_pazymius();
00087 friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00088 friend std::istream& operator>>(std::istream& is, Studentas& s);
00089 friend std::ofstream& operator<<(std::ofstream& os, const Studentas& s);
00090 };
00091
00092
00093
00094
00096 using StudentuGrupe = std::vector<Studentas>; // vector
00097 //using StudentuGrupe = std::list<Studentas>; // list
00098 //using StudentuGrupe = std::deque<Studentas>; // deque
00099 const int Maxpazymiu=20;
00100 const int Maxstudentu=10000000;

```

7.7 src/generavimas.cpp Failo Nuoroda

#include "funkcijos.h"

Įtraukimo priklausomybių diagrama generavimas.cpp:

Funkcijos

- void generavimasSk (Studentas &A, StudentuGrupe &grupe, int &pasirinkimas)
- void generavimasVisko (Studentas &A, StudentuGrupe &grupe, int &pasirinkimas)
- void failuGeneravimas (int &n)

7.7.1 Funkcijos Dokumentacija

7.7.1.1 failuGeneravimas()

```

void failuGeneravimas (
    int & n)

```

Apibrėžimas failo [generavimas.cpp](#) eilutėje 73.

Here is the caller graph for this function:

7.7.1.2 generavimasSk()

```

void generavimasSk (
    Studentas & A,
    StudentuGrupe & grupe,
    int & pasirinkimas)

```

Apibrėžimas failo [generavimas.cpp](#) eilutėje 3.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.7.1.3 generavimasVisko()

```
void generavimasVisko (
    Studentas & A,
    StudentuGrupe & grupe,
    int & pasirinkimas)
```

Inicijuojame atsitiktinių skaičių generatorių su dabartiniu laiku

`static_cast<long unsigned int>` naudojamas norint užtikrinti, kad laiko reikšmė būtų tinkamai konvertuota į generatoriaus sėklą

Sukuriame tolygų skaičių pasiskirstymą nuo 0 iki 9

Apibrėžimas failo [generavimas.cpp](#) eilutėje 35.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.8 generavimas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "funkcijos.h"
00002
00003 void generavimasSk(Studentas &A, StudentuGrupe &grupe, int &pasirinkimas)
00004 {
00005     int m = skaiciu_mastelis("Kiek yra studentų? ", 1, 1000);
00006     for(int i=0;i<m;i++)
00007     {
00008         A.setVardas(vardo_skaitymas("Įveskite studento vardą: "));
00009         A.setPavarde(vardo_skaitymas("Įveskite studento pavardę: "));
00010         cout<<"Iš viso gali būti įvesta «Maxpazymiu» pažymių."<<endl;
00011         cout<<"Įveskite semestro pažymius : \n";
00012         int n = skaiciu_mastelis("Kiek pažymių norite sugeneruoti? ", 1, Maxpazymiu);
00013         int temp;
00014         for(int ii=0;ii<n;ii++)
00015         {
00016             temp = rand() % 10 + 1; // Generuoja atsitiktinius skaičius nuo 1 iki 10
00017             cout<<"Sugeneruotas «ii+1» pažymys: " <<temp<<endl;
00018             A.setPaz(temp);
00019         }
00020         if(pasirinkimas==1 && n<Maxpazymiu)
00021         {
00022             for(int ii=0;ii<Maxpazymiu-n;ii++)
00023             {
00024                 A.setPaz(0);
00025             }
00026         }
00027
00028         A.setEgz(rand() % 10 + 1); // Generuoja atsitiktinius skaičius nuo 1 iki 10
00029         cout<<"Sugeneruotas egzamino pažymys: " <<A.getEgz()<<endl;
00030         A.skaiciuoti_rezultata(pasirinkimas);
00031         grupe.push_back(A);
00032         A.isvalyti_pazymius();
00033     }
00034 }
00035 void generavimasVisko(Studentas &A, StudentuGrupe &grupe, int &pasirinkimas)
00036 {
00037     mt19937 mt(static_cast<long unsigned
00038 int>(std::chrono::high_resolution_clock::now().time_since_epoch().count()));
00039     uniform_int_distribution<int> dist(0,9);
00040
00041     string vardai[10]={ "Jonas", "Petras", "Ona", "Maryte", "Antanas", "Ieva", "Tomas", "Rasa",
00042 "Dainius", "Asta"};
00043     string pavarde_m[10]={ "Pavardaitel1", "Pavardaitel2", "Pavardaitel3", "Pavardaitel4", "Pavardaitel5",
00044 "Pavardaitel6", "Pavardaitel7", "Pavardaitel8", "Pavardaitel9", "Pavardaitel10"};
00045     string pavardenis_v[10]={ "Pavardenis1", "Pavardenis2", "Pavardenis3", "Pavardenis4", "Pavardenis5",
00046 "Pavardenis6", "Pavardenis7", "Pavardenis8", "Pavardenis9", "Pavardenis10"};
00047     int m = skaiciu_mastelis("Kiek yra studentų? ", 1, 1000);
00048     cout<<"Iš viso gali būti įvesta «Maxpazymiu» pažymių."<<endl;
```

```

00046     cout<<"Kiek pažymių norite sugeneruoti? "<<endl;
00047     int n = skaiciu_mastelis("", 1, Maxpazymiu);
00048     for(int i=0;i<m;i++)
00049     {
00050         A.setVardas(vardai[dist(mt)]);
00051         switch(*A.getVardas().rbegin())//pasirenkame pavarde pagal vardo gala
00052         {
00053
00054             case 's': A.setPavarde(pavardes_v[dist(mt)]); break;
00055             default: A.setPavarde(pavardes_m[dist(mt)]); break;
00056         }
00057         int temp;
00058         for(int ii=0;ii<n;ii++)
00059         {
00060             temp = dist(mt)+1; // Generuoja atsitiktinius skaičius nuo 1 iki 10
00061             cout<<"Sugeneruotas "<<ii+1<<" pažymys: "<<temp<<endl;
00062             A.setPaz(temp);
00063         }
00064
00065         A.setEgz(dist(mt)+1); // Generuoja atsitiktinius skaičius nuo 1 iki 10
00066         cout<<"Sugeneruotas egzamino pažymys: "<<A.getEgz()<<endl;
00067         A.skaiciuoti_rezultata(pasirinkimas);
00068         grupe.push_back(A);
00069         A.isvalyti_pazymius();
00070     }
00071 }
00072 }
00073 void failuGeneravimas( int &n)
00074 {
00075     std::ofstream failas("studentai_"+std::to_string(n)+".txt");
00076     if (!failas.is_open()) {
00077         cout << "Nepavyko sukurti failo studentai_" << n << ".txt" << endl;
00078         return;
00079     }
00080 }
00081
00082     int m = 10; // Generuojame 10 pažymių kiekvienam studentui testui
00083     mt19937 mt(static_cast<long unsigned
int>(std::chrono::high_resolution_clock::now().time_since_epoch().count())); // Inicijuojame
atsitiktinių skaičių generatorių su dabartiniu laiku
00084     //static_cast<long unsigned int> naudojamas norint užtikrinti, kad laiko reikšmė būtų tinkamai
konvertuota į generatoriaus sėklą
00085     uniform_int_distribution<int> dist(0,9); // Sukuriame tolygų skaičių pasiskirstymą nuo 0 iki 9
00086
00087     string vardai[10]={ "Jonas", "Petras", "Ona", "Maryte", "Antanas", "Ieva", "Tomas", "Rasa",
"Dainius", "Asta"};
00088     string pavardes_m[10]={ "Pavardaitel1", "Pavardaitel2", "Pavardaitel3", "Pavardaitel4", "Pavardaitel5",
"Pavardaitel6", "Pavardaitel7", "Pavardaitel8", "Pavardaitel9", "Pavardaitel10"};
00089     string pavardes_v[10]={ "Pavardenis1", "Pavardenis2", "Pavardenis3", "Pavardenis4", "Pavardenis5",
"Pavardenis6", "Pavardenis7", "Pavardenis8", "Pavardenis9", "Pavardenis10"};
00090     failas<<left<<setw(15)<<"Vardas"<<left<<setw(15)<<"Pavarde";
00091     int a=45;
00092     for(int i=0; i<m;i++)
00093     {
00094         failas<<left<<setw(5)<<"ND"+std::to_string(i+1);
00095         a+=5;
00096     }
00097     failas<<left<<setw(5)<<"Egz.\n";
00098     for(int i=0;i<n;i++)
00099     {
00100         string vardas=vardai[dist(mt)];
00101         string pavarde;
00102         switch(*vardas.rbegin())//pasirenkame pavarde pagal vardo gala
00103         {
00104
00105             case 's': pavarde=pavardes_v[dist(mt)]; break;
00106             default: pavarde=pavardes_m[dist(mt)]; break;
00107         }
00108         failas<<left<<setw(15)<<vardas<<left<<setw(15)<<pavarde;
00109
00110         for(int j=0;j<m;j++)
00111         {
00112             int temp = dist(mt)+1; // generuojam pazymius nuo 1 iki 10
00113             failas<<left<<setw(5)<<temp;
00114         }
00115
00116         int egz = dist(mt)+1; // generuojam egzamino pazymi nuo 1 iki 10
00117         failas<<left<<setw(5)<<egz<<"\n";
00118     }
00119     failas.close();
00120 }

```

7.9 src/ived_isved.cpp Failo Nuoroda

```
#include "funkcijos.h"
Įtraukimo priklausomybių diagrama ived_isved.cpp:
```

Funkcijos

- [StudentuGrupe bufer_nusk](#) (string &read_vardas, int &pasirinkimas, int &m)
- void [inputas](#) ([Studentas](#) &A, [StudentuGrupe](#) &grupe, int &pasirinkimas)
- void [outputas](#) (const [StudentuGrupe](#) &vargsiukai, const [StudentuGrupe](#) &smartukai, int &pasirinkimas, int &isvedimas, int &m)

7.9.1 Funkcijos Dokumentacija

7.9.1.1 bufer_nusk()

```
StudentuGrupe bufer_nusk (
    string & read_vardas,
    int & pasirinkimas,
    int & m)
```

Sukuriame stringstream objektą, kuris leis mums lengvai išskaidyti eilutę į atskirus žodžius ir skaičius

Skaitome eilutę į [Studentas](#) objektą per klasės metodą

Išvalome pažymių vektorių, kad jis būtų tuščias prieš kitą studento įvedimą

Apibrėžimas failo [ived_isved.cpp](#) eilutėje 4.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.9.1.2 inputas()

```
void inputas (
    Studentas & A,
    StudentuGrupe & grupė,
    int & pasirinkimas)
```

Rankinis įvedimas per klasės metodą

Pridedame studentą į grupę

Išvalome pažymių vektorių, kad jis būtų tuščias prieš kitą studento įvedimą

Apibrėžimas failo [ived_isved.cpp](#) eilutėje 60.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.9.1.3 outputas()

```
void outputas (
    const StudentuGrupe & vargsiukai,
    const StudentuGrupe & smartukai,
    int & pasirinkimas,
    int & isvedimas,
    int & m)
```

Pradedame matuoti laiką

`std::chrono::high_resolution_clock::now()` funkcija grąžina dabartinį laiką, kuris bus naudojamas vėliau apskaičiuoti, kiek laiko užtruko failo nuskaitymas ir apdorojimas

Apskaičiuojame, kiek laiko praėjo nuo pradžios iki dabar, ir išsaugome šį laiką `diff` kintamajame

`std::chrono::duration<double>` yra tipas, kuris saugo laiką sekundėmis kaip `double` reikšmę, o `diff.count()` grąžina šią reikšmę, kurią mes išvedame į ekraną

Apskaičiuojame, kiek laiko praėjo nuo pradžios iki dabar, ir išsaugome šį laiką `diff` kintamajame

`std::chrono::duration<double>` yra tipas, kuris saugo laiką sekundėmis kaip `double` reikšmę, o `diff.count()` grąžina šią reikšmę, kurią mes išvedame į ekraną

Apibrėžimas failo `ived_isved.cpp` eilutėje 74.

Here is the caller graph for this function:

7.10 ived_isved.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "funkcijos.h"
00002
00003
00004 StudentuGrupe bufer_nusk(string &read_vardas,int &pasirinkimas, int &m)
00005 {
00006     StudentuGrupe grupe;// Sukuriame tuščią studentų grupės vektorių
00007     while(true)
00008     {
00009         try
00010         { m=0;
00011
00012             std::ifstream open_f(read_vardas);// Atidarome failą skaitymui
00013             if (!open_f.is_open())
00014             {
00015                 throw std::runtime_error("Nepavyko atidaryti failo " + read_vardas);//jeigu failas nebuvo
00016                 atidarytas, ismeta klaidą
00017             }
00018             // auto start = std::chrono::high_resolution_clock::now();// Pradedame matuoti laiką
00019             /* std::chrono::high_resolution_clock::now() funkcija grąžina dabartinį laiką,
00020             kuris bus naudojamas vėliau apskaičiuoti, kiek laiko užtruko failo nuskaitymas ir
00021             apdorojimas*/
00022             string eil;// Sukuriame eilutės kintamąjį, kuris bus naudojamas skaityti kiekvieną eilutę iš
00023             failo
00024             std::getline(open_f, eil);// Skaitome pirmąją eilutę, kuri yra antraštė, ir ją ignoruojame
00025             while (std::getline(open_f, eil)) // Skaitome kiekvieną likusią eilutę iš failo, kol
00026             pasiekiamė failo pabaigą
00027             {
00028                 if (eil.empty()) // Tikriname, ar eilutė yra tuščia, ir jei taip, praleidžiame ją
00029                 {
00030                     continue;
00031                 }
00032                 std::stringstream ss(eil);
00033                 Studentas A;
```

```

00033         ss>>A;
00034         if (A.getVardas().empty() || A.getPavarde().empty())
00035         {
00036             continue;
00037         }
00038
00039         A.skaiciuoti_rezultata(pasirinkimas);
00040
00041         A.isvalyti_pazymius();
00042
00043         grupe.push_back(A);
00044         m++;
00045     }
00046
00047     //std::chrono::duration<double> diff = std::chrono::high_resolution_clock::now() - start;//
    Apskaičiuojame, kiek laiko praėjo nuo pradžios iki dabar, ir išsaugome šį laiką diff kintamajame
00048     //std::chrono::duration<double> yra tipas, kuris saugo laiką sekundėmis kaip double reikšmę, o
    diff.count() grąžina šią reikšmę, kurią mes išvedame į ekraną
00049     //cout << "Failo nuskaitymas ir apdorojimas užtruko: " << diff.count() << " sekundziu." << endl;
00050     open_f.close();
00051     return grupe;
00052 }
00053 catch(const std::runtime_error& e)
00054 {
00055     cout<<"Klaida: "«e.what()«". Patikrinkite ar failas egzistuoja ir bandykite dar kartą. \n";
00056 }
00057 }
00058 }
00059 }
00060 void inputas(Studentas &A, StudentuGrupe &grupe, int &pasirinkimas)
00061 {
00062     int m = 1; // Pradinis studentų kiekis, nustatomas į 1, kad įvesties ciklas prasidėtų
00063
00064     while (m!=0)
00065     {
00066         A.nuskaityti_ranka(Maxpazymiu);
00067         A.skaiciuoti_rezultata(pasirinkimas);
00068         grupe.push_back(A);
00069         A.isvalyti_pazymius();
00070         cout<<"Jei norite ivesti dar viena studenta, iveskite 1, jei ne - 0: ";
00071         cin>>m;
00072     }
00073 }
00074 void outputas(const StudentuGrupe &vargsiukai, const StudentuGrupe &smartukai, int &pasirinkimas, int
    &isvedimas, int &m)
00075 {
00076
00077     auto start = std::chrono::high_resolution_clock::now();
00078     if(isvedimas==2)
00079     {
00080         std::ofstream out_f("vargsiukai"+std::to_string(m)+".txt");
00081         std::ofstream out_s("smartukai"+std::to_string(m)+".txt");
00082
00083         if(pasirinkimas==1)
00084         {
00085             out_f << left << setw(15) << "Vardas:" << left << setw(30) << "Pavardė:" << left << setw(45) <<
00086             "Galutinis(vid.):" << '\n';
00087             out_f << "
00088             -----"
00089             << '\n';
00090             out_f << fixed << setprecision(2);
00091             for (const auto &A : vargsiukai)
00092             {
00093                 out_f << A << '\n';
00094             }
00095             }
00096             else
00097             {
00098                 out_f << left << setw(15) << "Vardas:" << left << setw(30) << "Pavardė:" << left << setw(45) <<
00099                 "Galutinis(med.):" << '\n';
00100                 out_f << "
00101                 -----"
00102                 << '\n';
00103                 out_f << fixed << setprecision(2);
00104                 for (const auto &A : vargsiukai)
00105                 {
00106                     out_f << A << '\n';
00107                 }
00108                 if(pasirinkimas==1)
00109                 {
00110                     out_s << left << setw(15) << "Vardas:" << left << setw(30) << "Pavardė:" << left << setw(45) <<
00111                     "Galutinis(vid.):" << '\n';
00112                     out_s << "
00113                     -----"

```

```

00111     « '\n';
00112         out_s « fixed « setprecision(2);
00113         for (const auto &A : smartukai)
00114         {
00115             out_s « A « '\n';
00116         }
00117     }
00118     else
00119     {
00120         out_s « left « setw(15) « "Vardas:" « left « setw(30) « "Pavardė:" « left « setw(45) «
00121 "Galutinis(med.):" « '\n';
00122         out_s « "
-----"
00123     « '\n';
00124         out_s « fixed « setprecision(2);
00125         for (const auto &A : smartukai)
00126         {
00127             out_s « A « '\n';
00128         }
00129     }
00130
00131     cout<<"Rezultatai išsaugoti failuose vargsiukai"+std::to_string(m)+".txt ir
smartukai"+std::to_string(m)+".txt"<<endl;
00132     std::chrono::duration<double> diff = std::chrono::high_resolution_clock::now() - start;
00133     cout « "Duomenų išvedimas užtruko: " « diff.count() « " sekundžių." « endl;
00134     out_f.close();
00135     out_s.close();
00136
00137     return;
00138 }
00139
00140
00141
00142
00143     if (pasirinkimas == 1)
00144     {
00145         cout « left « setw(15) « "Vardas:" « left « setw(30) « "Pavardė:" « left « setw(45) «
00146 "Galutinis(vid.):" « '\n';
00147     }
00148     else
00149     {
00150         cout « left « setw(15) « "Vardas:" « left « setw(30) « "Pavardė:" « left « setw(45) «
00151 "Galutinis(med.):" « '\n';
00152     }
00153     cout « "
-----"
00154     « '\n';
00155     cout « "Vargsiukai:" « '\n';
00156     for (const auto &A : vargsiukai)
00157     {
00158         cout « A « '\n';
00159     }
00160     cout « '\n' « "Smartukai:" « '\n';
00161     for (const auto &A : smartukai)
00162     {
00163         cout « A « '\n';
00164     }
00165
00166     std::chrono::duration<double> diff = std::chrono::high_resolution_clock::now() - start;
00167     cout « "Duomenų išvedimas užtruko: " « diff.count() « " sekundžių." « endl;
00168 }

```

7.11 src/main.cpp Failo Nuoroda

```

#include "funkcijos.h"
#include <windows.h>
Įtraukimo priklausomybių diagrama main.cpp:

```

Funkcijos

- `int main ()`

7.11.1 Funkcijos Dokumentacija

7.11.1.1 main()

```
int main ()
```

Nustatome konsolės išvesties kodavimą į UTF-8, kad būtų galima teisingai rodyti lietuviškus simbolius

Nustatome konsolės įvesties kodavimą į UTF-8, kad būtų galima teisingai skaityti lietuviškus simbolius iš vartotojo įvesties

Išjungia sinchronizaciją tarp C++ srautų ir C srautų, kad pagerintų įvesties/išvesties našumą

Apibrėžimas failo [main.cpp](#) eilutėje 6.

Funkcijos kvietimo grafas:

7.12 main.cpp

Eiti į šio failo dokumentaciją.

```
00001
00002 #include "funkcijos.h"
00003 #include <windows.h> // Įtraukiame Windows.h biblioteką, kad galėtume naudoti funkcijas, susijusias su
                                konsolės kodavimo nustatymais
00004
00005
00006 int main() {
00007     SetConsoleOutputCP(CP_UTF8);
00008     SetConsoleCP(CP_UTF8);
00009
00010     system("powershell ls studentai*.txt");
00011     std::ios::sync_with_stdio(false);
00012     //std::ios::sync_with_stdio(false) funkcija yra naudojama norint pagerinti įvesties/išvesties
našumą,
00013     // išjungiant sinchronizaciją tarp C++ srautų (std::cin, std::cout) ir C srautų (scanf, printf).
00014     // Tai leidžia C++ srautams veikti greičiau, nes jie nebėra priversti sinchronizuotis su C
srautais.
00015     // Tačiau tai reiškia, kad negalima naudoti C srautų funkcijų kartu su C++ srautais, nes tai gali
sukelti nenumatytų rezultatų,
00016     // todėl rekomenduojama naudoti tik vieną tipą srautų visame programme. pvz. jei naudojate
std::cout, venkite naudoti printf ir atvirkščiai.
00017
00018
00019     Studentas A;
00020     StudentuGrupe grupe;
00021     StudentuGrupe vargsiukai;
00022     StudentuGrupe smartukai;
00023     int pasirinkimas;
00024     int isvedimas;
00025     int m;
00026     //Zmogus z;
00027     cout<<"Ką jūs norite padaryti? \n 1 - rankiniu būdu įvesti studentus \n 2 - generuoti tik pažymius
\n 3 - generuoti vardus su pažymiais \n 4 - nuskaityti iš failo \n 5 -baigti darbą \n 6 -generuoti
failą \n 7- testuoti generuojant failus(daryti kai nera failų) \n 8 - testuoti visa programa \n 9 -
testuoti klasės metodus \n Pasirinkite: ";
00028     int veiksmas = skaiciu_mastelis("", 1, 9);
00029     if(veiksmas==5) return 0;
00030     if(veiksmas==6)
00031     {int n =skaiciu_mastelis("Kiek yra studentų? ", 1, 100000);
00032     failuGeneravimas( n); return 0;}
00033     if(veiksmas==7) {tyrimasFailoKurimas(); return 0;}
00034     if(veiksmas==8) {tyrimasVisasProcesas(); return 0;}
00035     if(veiksmas==9) {tyrimasKlasesMetodams(); return 0;}
00036     cout<<"Kaip norite apskaičiuoti galutinį balą? \n 1 - pagal vidurkį \n 2 - pagal medianą \n
Pasirinkite: ";
00037     pasirinkimas = skaiciu_mastelis("", 1, 2);
00038     cout<<"Kaip norite rikiuoti rezultatus? \n 1 - pagal vardą \n 2 - pagal pavardę \n 3 - pagal
galutinį balą \n Pasirinkite: ";
00039     int rik = skaiciu_mastelis("", 1, 3);
00040     cout<<"Kaip norite išvesti rezultatus? \n 1 - į ekraną \n 2 - į failą \n Pasirinkite: ";
00041     isvedimas = skaiciu_mastelis("", 1, 2);
00042     if(veiksmas==1) inputas(A, grupe, pasirinkimas);
```

```

00043     else if(veiksmas==2) generavimasSk(A, grupe, pasirinkimas);
00044     else if(veiksmas==3) generavimasVisko(A, grupe, pasirinkimas);
00045     else if (veiksmas==4) {
00046         string read_vardas;
00047         cout << "Iveskite failo pavadinimą iš sarašo (pvz. studentai10000.txt): ";
00048         cin >> read_vardas;
00049         grupe = bufer_nusk( read_vardas,pasirinkimas, m);
00050     }
00051
00052
00053 int rus = skaiciu_mastelis("Pasirinkite studentu skirstymo strategija (1, 2 arba 3): ", 1, 3);
00054
00055
00056     rikiavimas(grupe, rik);
00057     studentoLygis(grupe, vargsiukai, smartukai, rus);
00058     outputas(vargsiukai, smartukai, pasirinkimas, isvedimas, m);
00059
00060     return 0;
00061 }
00062
00063

```

7.13 src/papildomos_funkcijos.cpp Failo Nuoroda

```
#include "funkcijos.h"
```

Įtraukimo priklausomybių diagrama papildomos_funkcijos.cpp:

Funkcijos

- void [rikiavimas](#) ([StudentuGrupe](#) &grupe, int &rik)
Funkcija rikiavimui, kuri naudoja lambda funkcijas kaip comparatorius, kad galėtų rikiuoti pagal vardą, pavardę arba galutinį balą, priklausomai nuo vartotojo pasirinkimo.
- int [skaiciu_mastelis](#) (const string &prompt, int min_val, int max_val)
- void [studentoLygis](#) ([StudentuGrupe](#) &grupe, [StudentuGrupe](#) &vargsiukai, [StudentuGrupe](#) &smartukai, int &rusiavimas)
- string [vardo_skaitymas](#) (const string &prompt)
- double [skaiciu_skaitymas](#) (const string &prompt, double min_val, double max_val)

7.13.1 Funkcijos Dokumentacija

7.13.1.1 rikiavimas()

```

void rikiavimas (
    StudentuGrupe & grupe,
    int & rik)

```

Funkcija rikiavimui, kuri naudoja lambda funkcijas kaip comparatorius, kad galėtų rikiuoti pagal vardą, pavardę arba galutinį balą, priklausomai nuo vartotojo pasirinkimo.

list naudos list::sort(), o vector/deque naudos std::sort()

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 14.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.13.1.2 skaiciu_mastelis()

```
int skaiciu_mastelis (
    const string & prompt,
    int min_val,
    int max_val)
```

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 30.

Here is the caller graph for this function:

7.13.1.3 skaiciu_skaitymas()

```
double skaiciu_skaitymas (
    const string & prompt,
    double min_val,
    double max_val)
```

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 156.

7.13.1.4 studentoLygis()

```
void studentoLygis (
    StudentuGrupe & grupe,
    StudentuGrupe & vargsiukai,
    StudentuGrupe & smartukai,
    int & rusiavimas)
```

1 strategija: visi studentai lieka grupes, kopijuojami į vargsiukai ir smartukai

2 strategija: vienu perėjimu vargsiukus perkeliame, o iš grupes ištriname.

3 strategija: dalijame su partition, vargsiukus perkeliame, grupėje paliekame tik smartukus.

Kopijuojame vargsiukus į atskirą konteinerį, naudojant std::copy ir std::back_inserter, kuris prideda elementus į vargsiukai konteinerio pabaigą.

Iš grupės ištriname vargsiukus, palikdami tik smartukus.

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 65.

Here is the caller graph for this function:

7.13.1.5 vardo_skaitymas()

```
string vardo_skaitymas (
    const string & prompt)
```

Kintamasis, kuris nurodo, ar įvestyje yra bent viena raidė

Kintamasis, kuris nurodo, ar visi įvesties simboliai yra raidės arba skaičiai

std::isalpha funkcija tikrina, ar simbolis yra raidė

std::isalnum funkcija tikrina, ar simbolis yra raidė arba skaičius

std::numeric_limits<std::streamsize>::max() - tai funkcija, kuri grąžina didžiausią galimą streamsize reikšmę, kuri yra naudojama kaip argumentas ignore funkcijai, kad būtų ignoruojami visi likę simboliai įvesties sraute iki naujos eilutės simbolio ('').

Apibrėžimas failo [papildomos_funkcijos.cpp](#) eilutėje 105.

Here is the caller graph for this function:

7.14 papildomos_funkcijos.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "funkcijos.h"
00002
00003 template <typename Container, typename Compare>
00004 static void rikiuoti(Container &grupe, Compare comp)
00005 {
00006     sort(grupe.begin(), grupė.end(), comp);
00007 }
00008 template <typename Compare>
00009 static void rikiuoti(std::list<Studentas> &grupe, Compare comp)
00010 {
00011     grupė.sort(comp);
00012 }
00014 void rikiavimas(StudentuGrupe &grupe, int &rik)
00015 {
00016     // Lambda comparatoriai
00017     auto compare_vardas = [](const Studentas &a, const Studentas &b) { return a.getVardas() >
00018         b.getVardas(); };
00018     auto compare_pavarde = [](const Studentas &a, const Studentas &b) { return a.getPavarde() >
00019         b.getPavarde(); };
00019     auto compare_rez = [](const Studentas &a, const Studentas &b) { return a.Rezultatas() >
00020         b.Rezultatas(); };
00020
00022     if (rik == 1) {
00023         rikiuoti(grupe, compare_vardas);
00024     } else if (rik == 2) {
00025         rikiuoti(grupe, compare_pavarde);
00026     } else if (rik == 3) {
00027         rikiuoti(grupe, compare_rez);
00028     }
00029 }
00030 int skaiciu_mastelis(const string &prompt, int min_val, int max_val)
00031 {
00032     int value;
00033     while (true) {
00034         try
00035         {
00036             cout<<prompt;
00037             cin >> value;
00038             if(cin.fail())
00039             {
00040                 throw std::runtime_error("Įvestas ne skaičius");
00041             }
00042             if (value < min_val || value > max_val) {
00043                 throw std::out_of_range("Įvestas skaičius už intervalo ribų");
00044             }
00045             return value;
00046         }
00047     }
00048     catch(const std::runtime_error& e)
00049     {
00050         cin.clear();
00051         cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00052         cout<<"Klaida: "«e.what()«". Bandykite dar karta.\n";
00053     }
00054     catch(const std::out_of_range& e)
00055     {
00056         cout<<"Klaida: "«e.what()«". Turi būti nuo "«min_val«" iki "«max_val«".\n";
00057     }
00058 }
00059
00060 void studentoLygis(StudentuGrupe &grupe, StudentuGrupe &vargsiukai, StudentuGrupe &smartukai, int
&rusiavimas)
00061 {
00062     // jeigu nori spausdinti i failus, komentuok situos
00063     vargsiukai.clear();
00064     smartukai.clear();
00065
00066     if (rusiavimas == 1)
00067     {
00068         for (const auto &A : grupė)
00069         {
00070             if (A.Rezultatas() < 5.0) vargsiukai.push_back(A);
00071             else smartukai.push_back(A);
00072         }
00073     }
00074     else if (rusiavimas == 2)
00075     {

```

```

00083         for(auto it = grupe.rbegin(); it != grupe.rend(); ++it) // Naudojame reverse iterator, kad
galėtume saugiai trinti elementus iš grupės, nes std::vector ir std::deque iteratoriaus invalidacija
įvyksta, kai triname elementus, o reverse iterator leidžia mums saugiai iteruoti atgal ir trinti
elementus be rizikos sugadinti iteratorių.
00084     {
00085         const auto &A = *it;
00086
00087         if(A.Rezultatas() < 5.0) {vargsiukai.push_back(A); grupe.pop_back();}
00088     }
00089     smartukai = grupe; // likusi grupė yra smartukai
00090 }
00091 else if( rusiavimas == 3)
00092 {
00093     auto border = std::partition(grupe.begin(), grupe.end(), [](const Studentas &A) { return
A.Rezultatas() >= 5.0; });
00094
00095     std::copy(border, grupe.end(), std::back_inserter(vargsiukai));
00096     grupe.erase(border, grupe.end());
00097     smartukai = grupe;
00098 }
00099 //jeigu nori, kad nespausdintu i failus, komentuok situos
00100 //vargsiukai.clear();
00101 //smartukai.clear();
00102
00103 }
00104 }
00105 string vardo_skaitymas(const string &prompt)
00106 {
00107     string value;
00108     while (true) {
00109         try
00110         {
00111             cout<<prompt;
00112             cin>>value;
00113             if(cin.fail())
00114             {
00115                 throw std::runtime_error("Įvesties klaida");
00116             }
00117
00118             bool has_alpha = false;
00119             bool all_alnum = true;
00120
00121             for(unsigned char ch : value)
00122             {
00123                 if(std::isalpha(ch))
00124                 {
00125                     has_alpha = true;
00126                 }
00127                 else if(!std::isalnum(ch))
00128                 {
00129                     all_alnum = false;
00130                     break;
00131                 }
00132             }
00133             if(!all_alnum || !has_alpha)
00134             {
00135                 throw std::invalid_argument("Turi būti bent viena raidė ir tik raidės bei skaičiai");
00136             }
00137         }
00138         return value;
00139     }
00140 }
00141 catch(const std::runtime_error& e)
00142 {
00143     cin.clear();
00144     cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00145     cout<<"Klaida: "«e.what()«". Bandykite dar karta.\n";
00146 }
00147 catch(const std::invalid_argument& e)
00148 {
00149     cout<<"Klaida: "«e.what()«". Bandykite dar karta.\n";
00150 }
00151 }
00152 }
00153 }
00154 }
00155 }
00156 double skaiciu_skaitymas(const string &prompt, double min_val, double max_val)
00157 {
00158     double value;
00159     while (true) {
00160         try
00161         {
00162             cout<<prompt;
00163             cin >> value;
00164             if(cin.fail())
00165             {

```

```

00166         throw std::runtime_error("Įvestas ne skaičius");
00167     }
00168     if (value < min_val || value > max_val) {
00169         throw std::out_of_range("Įvestas skaičius už intervalo ribų");
00170     }
00171     }
00172     return value;
00173 }
00174
00175 catch(const std::runtime_error& e)
00176 {
00177     cin.clear();
00178     cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00179     cout<<"Klaida: " <<e.what()<<". Bandykite dar karta.\n";
00180
00181 }
00182 catch(const std::out_of_range& e)
00183 {
00184     cout<<"Klaida: " <<e.what()<<". Turi būti nuo " <<min_val<< " iki " <<max_val<< ".\n";
00185
00186 }
00187
00188 }
00189
00190 }

```

7.15 src/studentas.cpp Failo Nuoroda

#include "studentas.h"
Įtraukimo priklausomybių diagrama studentas.cpp:

Funkcijos

- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)
isvedimo operatorius
- std::ofstream & [operator<<](#) (std::ofstream &os, const [Studentas](#) &s)
isvedimo operatorius i faila
- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
ivedimo operatorius

7.15.1 Funkcijos Dokumentacija

7.15.1.1 operator<<() [1/2]

```
std::ofstream & operator<< (
    std::ofstream & os,
    const Studentas & s)

```

isvedimo operatorius i faila

Apibrėžimas failo [studentas.cpp](#) eilutėje 182.

7.15.1.2 operator<<() [2/2]

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s)

```

isvedimo operatorius

Apibrėžimas failo [studentas.cpp](#) eilutėje 174.

7.15.1.3 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s)
```

ivedimo operatorius

Apibrėžimas failo [studentas.cpp](#) eilutėje 190.

7.16 studentas.cpp

[Eiti į šio failo dokumentaciją.](#)

```
00001 #include "studentas.h"
00002
00004 Studentas::Studentas() : Zmogus{"nepriskirtas", "nepriskirtas"}
00005 {
00006     //Vardas = "nepriskirtas";
00007     //Pavarde = "nepriskirtas";
00008     paz.clear();
00009     egz = 0; vidurkis = 0.0;
00010     mediana = 0.0; rez = 0.0;
00011     lygis = "nepriskirtas";
00012     //cout<<"Konstruktorius suveike\n";
00013 };
00015 Studentas::~Studentas()
00016 {
00017     Vardas = ""; Pavarde = ""; paz.clear(); egz = 0; vidurkis = 0.0; mediana = 0.0; rez = 0.0; lygis =
00018     "";
00019 };
00024 Studentas::Studentas(const Studentas& s) : Zmogus{s.Vardas, s.Pavarde},
00025     //Vardas{s.Vardas},
00026     //Pavarde{s.Pavarde},
00027     paz{s.paz},
00028     egz{s.egz},
00029     vidurkis{s.vidurkis},
00030     mediana{s.mediana},
00031     rez{s.rez},
00032     lygis{s.lygis}
00033 {
00034
00035 }
00039 Studentas::Studentas(Studentas&& s) : Zmogus{std::move(s.Vardas), std::move(s.Pavarde)},
00040     //Vardas{std::move(s.Vardas)},
00041     //Pavarde{std::move(s.Pavarde)},
00042     paz{std::move(s.paz)},
00043     egz{s.egz},
00044     vidurkis{s.vidurkis},
00045     mediana{s.mediana},
00046     rez{s.rez},
00047     lygis{std::move(s.lygis)}
00048 {
00049     s.Vardas.clear();
00050     s.Pavarde.clear();
00051     s.paz.clear();
00052     s.egz = 0;
00053     s.vidurkis = 0.0;
00054     s.mediana = 0.0;
00055     s.rez = 0.0;
00056     s.lygis.clear();
00057 }
00058
00060 Studentas& Studentas::operator=(const Studentas& s)
00061 {
00062     if(&s == this) return *this;
00063     Vardas = s.Vardas;
00064     Pavarde = s.Pavarde;
00065     paz = s.paz;
00066     egz = s.egz;
00067     vidurkis = s.vidurkis;
00068     mediana = s.mediana;
00069     rez = s.rez;
00070     lygis = s.lygis;
00071     return *this;
00072 }
00074 Studentas& Studentas::operator=(Studentas &&s)
```

```

00075 {
00076 if(&s == this) return *this;
00077 Vardas = std::move(s.Vardas);
00078 Pavarde = std::move(s.Pavarde);
00079 paz = std::move(s.paz);
00080 egz = s.egz;
00081 vidurkis = s.vidurkis;
00082 mediana = s.mediana;
00083 rez = s.rez;
00084 lygis = std::move(s.lygis);
00085 s.Vardas.clear();
00086 s.Pavarde.clear();
00087 s.paz.clear();
00088 s.egz = 0;
00089 s.vidurkis = 0.0;
00090 s.mediana = 0.0;
00091 s.rez = 0.0;
00092 s.lygis.clear();
00093
00094 return *this;
00095 }
00096
00097 void Studentas::nuskaityti_ranka(int max_pazymiu)
00098 {
00099     Vardas = vardo_skaitymas("Įveskite studento vardą: ");
00100     Pavarde = vardo_skaitymas("Įveskite studento pavardę: ");
00101
00102     paz.clear();
00103     cout << "Iš viso gali būti įvesta " << max_pazymiu << " pažymių." << endl;
00104     cout << "Įveskite semestro pažymius: " << endl;
00105
00106     int n = 1;
00107     int temp = 0;
00108     int k = 1;
00109
00110     while (k != 0 && n <= max_pazymiu) {
00111         temp = skaiciu_mastelis("Įveskite " + std::to_string(n) + " pažymį: ", 1, 10);
00112         paz.push_back(temp);
00113
00114         cout << "Jei norite įvesti dar vieną pažymį, įveskite 1, jei ne - 0: ";
00115         k = skaiciu_mastelis("", 0, 1);
00116         ++n;
00117     }
00118
00119     egz = skaiciu_mastelis("Įveskite egzamino pažymį: ", 1, 10);
00120 }
00121 void Studentas::nuskaityti_is_eilutes(std::istream& is)
00122 {
00123     paz.clear();
00124     is >> Vardas >> Pavarde;
00125
00126     int pazymys = 0;
00127     while (is >> pazymys) {
00128         paz.push_back(pazymys);
00129     }
00130
00131     if (!paz.empty()) {
00132         egz = paz.back();
00133         paz.pop_back();
00134     }
00135 }
00136 void Studentas::skaiciuoti_rezultata(int pasirinkimas)
00137 {
00138     if (paz.empty()) {
00139         rez = 0.0;
00140         return ;
00141     }
00142
00143     if (pasirinkimas == 1) {
00144         int suma = 0;
00145         for (int pazymys : paz) {
00146             suma += pazymys;
00147         }
00148         rez = suma * 1.0 / (1.0*paz.size()) * 0.4 + egz * 0.6;
00149
00150         return;
00151     }
00152
00153     sort(paz.begin(), paz.end());
00154     for(int pazymys : paz)
00155     {
00156         if(pazymys==0)
00157         {
00158             paz.erase(std::remove(paz.begin(), paz.end(), pazymys), paz.end());
00159         }
00160     }
00161 }

```

```

00162     if (paz.size() % 2 == 1) {
00163         mediana = paz[paz.size() / 2];
00164     } else {
00165         mediana = (paz[paz.size() / 2 - 1] + paz[paz.size() / 2]) / 2.0;
00166     }
00167     rez = mediana * 0.4 + egz * 0.6;
00168 }
00169 void Studentas::isvalyti_pazymius()
00170 {
00171     paz.clear();
00172 }
00174 std::ostream& operator<<(std::ostream& os, const Studentas& s)
00175 {
00176     os << left << setw(15) << s.Vardas
00177         << left << setw(30) << s.Pavarde
00178         << left << setw(45) << fixed << setprecision(2) << s.rez;
00179     return os;
00180 }
00182 std::ofstream& operator<<(std::ofstream& os, const Studentas& s)
00183 {
00184     os << left << setw(15) << s.Vardas << "|"
00185         << left << setw(30) << s.Pavarde << "|"
00186         << left << setw(45) << fixed << setprecision(2) << s.rez;
00187     return os;
00188 }
00190 std::istream& operator>>(std::istream& is, Studentas& s)
00191 {
00192     is >> s.Vardas >> s.Pavarde;
00193
00194     s.paz.clear();
00195     int pazymys;
00196
00197     while (is >> pazymys) {
00198         if(pazymys == 0) break; // Jei įvedamas 0, laikome, kad pažymių įvedimas baigtas
00199         s.paz.push_back(pazymys);
00200     }
00201
00202     if (!s.paz.empty()) {
00203         s.egz = s.paz.back();
00204         s.paz.pop_back();
00205     }
00206
00207     return is;
00208 }
00209 }

```

7.17 src/tyrimai.cpp Failo Nuoroda

#include "funkcijos.h"
 Įtraukimo priklausomybių diagrama tyrimai.cpp:

Funkcijos

- void [tyrimasKlasesMetodams\(\)](#)
- void [tyrimasFailoKurimas\(\)](#)
- void [tyrimasVisasProcesas\(\)](#)

7.17.1 Funkcijos Dokumentacija

7.17.1.1 tyrimasFailoKurimas()

```
void tyrimasFailoKurimas ()
```

Apibrėžimas failo [tyrimai.cpp](#) eilutėje 57.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.17.1.2 tyrimasKlasesMetodams()

```
void tyrimasKlasesMetodams ()
```

1. Copy konstruktoriaus testas

1. Copy priskyrimo operatoriaus testas

1. Move konstruktoriaus testas

1. Move priskyrimo operatoriaus testas

Apibrėžimas failo [tyrimai.cpp](#) eilutėje 3.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.17.1.3 tyrimasVisasProcesas()

```
void tyrimasVisasProcesas ()
```

Apibrėžimas failo [tyrimai.cpp](#) eilutėje 72.

Funkcijos kvietimo grafas: Here is the caller graph for this function:

7.18 tyrimai.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "funkcijos.h"
00002
00003 void tyrimasKlasesMetodams()
00004 {
00005     Studentas s1;
00006     s1.setVardas("Jonas");
00007     s1.setPavarde("Jonaitis");
00008     s1.setPaz(6);
00009     s1.setEgz(9);
00010     s1.skaiciuoti_rezultata(1); // Apskaičiuojame rezultatą pagal vidurkį
00011
00012     Studentas s2;
00013     cout << "Įveskite studento vardą, pavardę, pažymius ir egzamino pažymį(paskutinis ivestas pazymis
bus egzamino pazymys, o norint nutraukti ivedima iveskite 0):\n";
00014     cin>>s2;
00015     s2.skaiciuoti_rezultata(2); // Apskaičiuojame rezultatą pagal medianą
00016
00017     std::cout << "Pradinis studentas s1:\n";
00018     std::cout << s1 << "\n";
00019     std::cout << "Pradinis studentas s2:\n";
00020     std::cout << s2 << "\n";
00021
00022     Studentas s3{s1};
00023     std::cout << "Po copy konstruktoriaus (s3{s1}):\n";
00024     std::cout << "s1: " << s1 << "\n";
00025     std::cout << "s3: " << s3 << "\n";
00026
00027     Studentas s4;
00028     s4 = s2;
00029     std::cout << "Po copy priskyrimo (s4 = s2):\n";
00030     std::cout << "s2: " << s2 << '\n';
00031     std::cout << "s4: " << s4 << "\n";
00032
00033     Studentas s5{std::move(s1)};
00034     std::cout << "Po move konstruktoriaus (s5{std::move(s1)}):\n";
```

```

00037     std::cout << "s5: " << s5 << "\n\n";
00038     std::cout<<"s1"s1<<"\n\n"; // s1 turėtų būti tuščias arba "nulintas" po move konstruktoriaus
00039
00040
00042     Studentas s6;
00043     s6 = std::move(s2);
00044     std::cout << "Po move priskyrimo (s6 = std::move(s2)):\n";
00045     std::cout << "s6: " << s6 << "\n\n";
00046
00047     Studentas s7;
00048     s7.setVardas("Asta");
00049     s7.setPavarde("Astaitė");
00050     s7.whoAmI();
00051     // Zmogus z1{"Tomas", "Tomaitis"};
00052     // std::cout << "Zmogus z1:\n";
00053     // std::cout << "Vardas: " << z1.getVardas() << ", Pavardė: " << z1.getPavarde() << "\n";
00054
00055 }
00056
00057 void tyrimasFailoKurimas() {
00058     vector<int> dydziai = {100000, 1000000};
00059
00060     for (int n : dydziai)
00061     {
00062         string failas = "studentai_" + std::to_string(n) + ".txt";
00063
00064         auto pradzia = std::chrono::high_resolution_clock::now();
00065         failuGeneravimas(n);
00066         auto pabaiga = std::chrono::high_resolution_clock::now();
00067
00068         std::chrono::duration<double> trukme = pabaiga - pradzia;
00069         cout << failas << " kurimo laikas: " << trukme.count() << " s\n";
00070     }
00071 }
00072 void tyrimasVisasProcesas() {
00073     vector<int> dydziai = { 100000, 1000000};
00074
00075     for (int n : dydziai)
00076     {
00077
00078         string failas = "studentai_" + std::to_string(n) + ".txt";
00079         int pasirinkimas = 1;
00080         int rikiavimo_budas = 3;
00081         int isvedimas = 2;
00082         int rusiavimas = 3;
00083
00084         auto pradzia = std::chrono::high_resolution_clock::now();
00085         StudentuGrupe grupe = bufer_nusk(failas, pasirinkimas, n); // Nuskaitome duomenis iš failo
00086         auto pabaiga1 = std::chrono::high_resolution_clock::now();
00087         std::chrono::duration<double> trukme = pabaiga1 - pradzia;
00088         cout << failas << " nuskaitymo ir apdorojimo laikas: " << trukme.count() << " s\n";
00089         rikiavimas(grupe, rikiavimo_budas); // Rikiuojame pagal galutinį balą
00090         auto pabaiga2 = std::chrono::high_resolution_clock::now();
00091         std::chrono::duration<double> rikiavimo_trukme = pabaiga2 - pabaiga1;
00092         cout << failas << " rikiavimo laikas mazejimo tvarka su sort funkcija: " <<
            rikiavimo_trukme.count() << " s\n";
00093
00094         StudentuGrupe vargsiukai;
00095         StudentuGrupe smartukai;
00096         studentoLygis(grupe, vargsiukai, smartukai, rusiavimas); // Skirstome į vargšus ir smartukus
00097         auto pabaiga3 = std::chrono::high_resolution_clock::now();
00098         std::chrono::duration<double> studentoLygio_trukme = pabaiga3 - pabaiga2;
00099         cout << failas << " studentų lygio nustatymo laikas: " << studentoLygio_trukme.count() << " s\n";
00100
00101         outputas(vargsiukai, smartukai, pasirinkimas, isvedimas, n); // Išvedame į failus
00102         auto pabaiga4 = std::chrono::high_resolution_clock::now();
00103         std::chrono::duration<double> isvedimo_trukme = pabaiga4 - pabaiga3;
00104         cout << failas << " rezultatu isvedimo i faila laikas: " << isvedimo_trukme.count() << " s\n";
00105         std::chrono::duration<double> visa_trukme = pabaiga4 - pradzia;
00106         cout<<" \n";
00107         cout << failas << " visas proceso laikas: " << visa_trukme.count() << " s\n";
00108         cout << "-----\n";
00109     }
00110 }

```


Rodyklė

- ~Studentas
 - Studentas, [13](#)
- ~Zmogus
 - Zmogus, [18](#)
- bufer_nusk
 - funkcijos.h, [22](#)
 - ived_isved.cpp, [33](#)
- failuGeneravimas
 - funkcijos.h, [22](#)
 - generavimas.cpp, [30](#)
- funkcijos.h
 - bufer_nusk, [22](#)
 - failuGeneravimas, [22](#)
 - generavimasSk, [23](#)
 - generavimasVisko, [23](#)
 - inputas, [23](#)
 - outputas, [23](#)
 - rikiavimas, [24](#)
 - skaiciu_mastelis, [24](#)
 - skaiciu_skaitymas, [24](#)
 - studentoLygis, [25](#)
 - tyrimasFailoKurimas, [25](#)
 - tyrimasKlasesMetodams, [25](#)
 - tyrimasVisasProcesas, [25](#)
 - vardo_skaitymas, [26](#)
- generavimas.cpp
 - failuGeneravimas, [30](#)
 - generavimasSk, [30](#)
 - generavimasVisko, [30](#)
- generavimasSk
 - funkcijos.h, [23](#)
 - generavimas.cpp, [30](#)
- generavimasVisko
 - funkcijos.h, [23](#)
 - generavimas.cpp, [30](#)
- getEgz
 - Studentas, [13](#)
- getPavarde
 - Zmogus, [18](#)
- getPaz
 - Studentas, [13](#)
- getVardas
 - Zmogus, [18](#)
- include Directorijos aprašas, [9](#)
- include/bibliotekos.h, [21](#)
- include/funkcijos.h, [22](#), [26](#)
- include/studentas.h, [27](#), [29](#)
- inputas
 - funkcijos.h, [23](#)
 - ived_isved.cpp, [33](#)
- isvalyti_pazymius
 - Studentas, [14](#)
- ived_isved.cpp
 - bufer_nusk, [33](#)
 - inputas, [33](#)
 - outputas, [33](#)
- main
 - main.cpp, [37](#)
- main.cpp
 - main, [37](#)
- Maxpazymiu
 - studentas.h, [28](#)
- Maxstudentu
 - studentas.h, [28](#)
- nuskaityti_is_eilutes
 - Studentas, [14](#)
- nuskaityti_ranka
 - Studentas, [14](#)
- operator<<
 - Studentas, [17](#)
 - studentas.cpp, [42](#)
 - Zmogus, [19](#)
- operator>>
 - Studentas, [17](#)
 - studentas.cpp, [42](#)
- operator=
 - Studentas, [14](#)
- outputas
 - funkcijos.h, [23](#)
 - ived_isved.cpp, [33](#)
- papildomos_funkcijos.cpp
 - rikiavimas, [38](#)
 - skaiciu_mastelis, [38](#)
 - skaiciu_skaitymas, [39](#)
 - studentoLygis, [39](#)
 - vardo_skaitymas, [39](#)
- Pavarde
 - Zmogus, [19](#)
- readStudent
 - Studentas, [15](#)
- Rezultatas

- Studentas, 15
- rikiavimas
 - funkcijos.h, 24
 - papildomos_funkcijos.cpp, 38
- setEgz
 - Studentas, 15
- setPavarde
 - Studentas, 15
- setPaz
 - Studentas, 15
- setVardas
 - Studentas, 16
- skaiciu_mastelis
 - funkcijos.h, 24
 - papildomos_funkcijos.cpp, 38
 - studentas.h, 27
- skaiciu_skaitymas
 - funkcijos.h, 24
 - papildomos_funkcijos.cpp, 39
- skaiciuoti_rezultata
 - Studentas, 16
- src Directorijos aprašas, 9
- src/generavimas.cpp, 30, 31
- src/ived_isved.cpp, 33, 34
- src/main.cpp, 36, 37
- src/papildomos_funkcijos.cpp, 38, 40
- src/studentas.cpp, 42, 43
- src/tyrimai.cpp, 45, 46
- Studentas, 11
 - ~Studentas, 13
 - getEgz, 13
 - getPaz, 13
 - isvalyti_pazymius, 14
 - nuskaityti_is_eilutes, 14
 - nuskaityti_ranka, 14
 - operator<<, 17
 - operator>>, 17
 - operator=, 14
 - readStudent, 15
 - Rezultatas, 15
 - setEgz, 15
 - setPavarde, 15
 - setPaz, 15
 - setVardas, 16
 - skaiciuoti_rezultata, 16
 - Studentas, 12, 13
 - whoAml, 16
- studentas
 - studentas.h, 28
- studentas.cpp
 - operator<<, 42
 - operator>>, 42
- studentas.h
 - Maxpazymiu, 28
 - Maxstudentu, 28
 - skaiciu_mastelis, 27
 - studentas, 28
 - StudentuGrupe, 27
 - vardo_skaitymas, 27
- studentoLygis
 - funkcijos.h, 25
 - papildomos_funkcijos.cpp, 39
- StudentuGrupe
 - studentas.h, 27
- tyrimai.cpp
 - tyrimasFailoKurimas, 45
 - tyrimasKlasesMetodams, 45
 - tyrimasVisasProcesas, 46
- tyrimasFailoKurimas
 - funkcijos.h, 25
 - tyrimai.cpp, 45
- tyrimasKlasesMetodams
 - funkcijos.h, 25
 - tyrimai.cpp, 45
- tyrimasVisasProcesas
 - funkcijos.h, 25
 - tyrimai.cpp, 46
- Vardas
 - Zmogus, 19
- vardo_skaitymas
 - funkcijos.h, 26
 - papildomos_funkcijos.cpp, 39
 - studentas.h, 27
- whoAml
 - Studentas, 16
 - Zmogus, 19
- Zmogus, 17
 - ~Zmogus, 18
 - getPavarde, 18
 - getVardas, 18
 - operator<<, 19
 - Pavarde, 19
 - Vardas, 19
 - whoAml, 19
 - Zmogus, 18